Internship reports 4th year

Alexandre HERVE

4th year DIA, Ux/Design track

Thales SIX GTS, 2 rue Marcel Dassault 78140 Vélizy Villacoublay

14 April – 14 August

STAGIAIRE Ingénieur développement informatique

Acknowledgments

I would like to express my sincere gratitude to **Jerôme Bégon**, whose support and recommendation helped me secure this internship opportunity. I extend my thanks to **Louis-Philippe Henriques**, Technical Lead of NCOP, and **Aldo Maalouf**, AI Architect, for their trust, guidance, and technical mentorship throughout the project. Their support and feedback were essential to the success of this work. I am also grateful to my academic supervisor, **Fatima-Zahra Kaghat**, for her valuable insights, availability, and continued support during the entire internship.

Glossary and Abbreviations

- **NCOP**: *NATO Common Operational Picture* — a software platform for visualizing and managing operational data across allied forces.
- **NLP**: *Natural Language Processing* — a field of artificial intelligence that focuses on enabling machines to understand, interpret, and generate human language.
- **COP**: *Common Operational Picture* — operational files used in NCOP to describe and share situational awareness of BSO data.
- **BSO**: *Battle Space Object* — a unit or asset in the operational environment, described by attributes such as name, coordinates, and military classification.

Table of Contents

Table des matières

Abstract

This internship was centered around the development of a new feature for the **NCOP** (NATO Common Operational Picture) platform, leveraging artificial intelligence and more specifically **natural language processing (NLP)**. The core objective was to introduce a novel method for filtering BSOs (Battle Space Objects) using natural language queries, in contrast to the existing system which required users to manually navigate complex menus and have prior technical knowledge of the software.

My task was to design and implement a new interface that would allow users to input requests in plain language and have them automatically translated into filtering parameters. To achieve this, I worked on several interconnected components: the creation of structured datasets and an ontology based on **App6a military symbology**, the development of filtering mechanisms that operated on available COP data, the implementation of a backend API to interface with NCOP, and finally, the training of a transformer-based NLP model capable of converting natural language into **SPARQL** queries.

Throughout the internship, I was supported by two supervisors: **Aldo Maalouf**, who guided the AI-related components, and **Louis-Philippe Henriques**, who provided oversight on integration with the NCOP software. While I worked autonomously on most development tasks, I received assistance from a senior developer during the final implementation stages.

The project was structured in phases, starting with resource and data generation, followed by rule-based querying, backend integration, and finally model training. While the initial aim was to use SPARQL for querying, the complexity of the syntax led us to transition to **SQL**,

which resulted in improved performance and usability. The final system successfully demonstrated the feasibility of NLP-driven BSO filtering within operational defense software.

1. Professional Context of the Internship

1.1 The Defence Sector: Structure, Evolution, and Sustainability Commitments

My internship took place within the defense sector—a technologically sophisticated and strategically essential industry that supports national sovereignty, international peacekeeping, and rapid response operations. This sector spans a broad spectrum of capabilities, from traditional hardware and weapons systems to emerging technologies such as artificial intelligence, secure communications, cyber-defense, and space-based assets. Organizations within the defense sector operate under unique constraints, including long development cycles, rigorous compliance requirements, and high sensitivity due to the classified nature of their work.

The landscape of defense technology is continuously evolving, driven by geopolitical shifts, alliances like **NATO**, and the increasing need for interoperability across multinational forces. Leading defense contractors such as **Lockheed Martin**, **BAE Systems**, **Raytheon Technologies**, **Thales Group**, and **Airbus Defence and Space** compete and collaborate globally, offering solutions in avionics, intelligence systems, secure communications, and integrated command platforms. Innovations in AI, real-time decision support, and data fusion have become cornerstones of next-generation military systems.

However, the sector also faces rising expectations for sustainability and ethical governance. Given the resource-intensive nature of defense manufacturing and operations, there is growing emphasis on **Corporate Social Responsibility (CSR)**. For instance, **Thales** has adopted environmentally conscious manufacturing, aims to reduce emissions, and has aligned its CSR goals with the **UN Sustainable Development Goals (SDGs)**. The company also promotes responsible AI development, gender and cultural diversity, and health and safety in its workforce.

The defense sector’s transformation toward sustainability and ethical innovation reflects a broader understanding: that security providers must not only protect global interests but also operate in alignment with environmental, social, and ethical imperatives.

1.2 Positioning of Thales SIX in the Defence Sector

Within this global context, **Thales Group** distinguishes itself as a world leader in defense, aerospace, and security technologies. Operating in over 60 countries, Thales develops

solutions for both government and civilian customers, ranging from avionics and secure communications to cybersecurity and space infrastructure.

My internship was conducted within **Thales SIX GTS** (Systems and Infrastructure for Secure Communications and Information Management), a specialized division responsible for the design and delivery of secure, interoperable systems. Thales SIX plays a central role in enabling mission-critical information sharing across ministries of defense, NATO allies, and emergency services. Its primary focus is on the development of sovereign, secure, and interoperable architectures that support both strategic command and field-level operations.

One of Thales SIX's flagship contributions is to the **NCOP** program, which aims to provide NATO forces with a real-time **Common Operational Picture**. This software platform enables situational awareness by visualizing tactical, geographic, and strategic data in an integrated interface. Thales contributes to NCOP by providing advanced AI modules, secure data handling capabilities, and deep system integration knowledge. These systems are built to handle massive volumes of operational data while maintaining responsiveness and usability in fast-paced environments.

What makes Thales SIX particularly competitive is its strategic focus on **sovereignty**, **interoperability**, and **responsible innovation**. The division invests heavily in research and development (over 20% of annual revenues), with a particular emphasis on AI, secure cloud architectures, and quantum technologies. In doing so, it maintains technological leadership while reinforcing trust with international partners. It also emphasizes ethical AI development, ensuring that transparency, human oversight, and international legal standards are embedded in its products from the design phase.

1.3 My Role as an Intern at Thales SIX

My internship at Thales SIX offered an immersive and hands-on experience within a high-stakes technical environment. I was assigned to the AI engineering team and tasked with building a natural language interface that could simplify how users interact with the NCOP system's BSO data. Rather than assisting with minor components, I was responsible for the design, development, and partial integration of a complete filtering system driven by NLP models. This required me to apply not only programming and data processing skills, but also to understand the operational constraints and standards of the defense sector.

The work required strict adherence to internal security protocols and development standards. All code and documentation followed procedures for traceability, auditability, and secure deployment. I gained exposure to the broader software development process within Thales, including internal review cycles, component integration, and documentation practices. While I worked independently on most technical tasks, I regularly interacted with senior developers and project leads, especially during code integration phases and for performance validation.

Throughout the internship, I was granted significant autonomy. I designed the data pipeline, built the ontology from scratch, and fine-tuned the NLP model with a custom dataset. My contributions were not isolated prototypes; they were embedded in an operational context with real users and real-world data. I also engaged in technical discussions around responsible AI usage, including model transparency, user control, and ethical limits in

defense applications. These conversations deepened my understanding of what it means to build AI systems in sensitive, high-impact domains.

Additionally, I had the opportunity to participate in regular exchanges with a **Belgian team** developing a similar C4ISR (Command, Control, Communications, Computers, Intelligence, Surveillance and Reconnaissance) solution. Their interest in the project highlighted the broader relevance of the NLP interface I was building, and our meetings provided valuable perspectives on system generalization, performance benchmarking, and future interoperability across NATO systems.

Overall, the internship was a deeply formative experience. It sharpened my technical skills, exposed me to the real-world constraints of secure software development, and allowed me to contribute to a system of strategic importance. Most importantly, it confirmed my desire to work at the intersection of AI and high-impact, operational domains.

2. INTERNSHIP PRESENTATION

2.1 Objectives of the Internship and Assigned Projects

The main objective of my internship was to design and implement a natural language processing (NLP) pipeline that would allow users of the NCOP (NATO Common Operational Picture) platform to apply filters to BSOs (Blue Situational Objects) through natural language queries. The goal was to replace the current filtering mechanism, which relied on manually selecting parameters or understanding structured symbology codes, with a more intuitive and efficient interface based on everyday language. This would make the platform more accessible, especially during time-sensitive operations where quick situational awareness is critical.

To achieve this, my work focused on several key components. First, I had to build foundational resources, such as a complete decoding system for App6a symbology. This required creating comprehensive CSV files that mapped the structure of these military codes, including all their possible variants, and building an ontology in OWL format to represent the hierarchical relationships between various military units and symbols. The ontology was necessary to allow semantic interpretation of user queries, especially when dealing with categories that had subtypes or nested meanings.

The next objective involved implementing a system that could take a user’s request in natural language, extract the relevant keywords, and match them to the correct categories or filters. Initially, I explored the use of SPARQL queries to query data directly from XML and CDF files, and I fine-tuned a T5-based model to generate these queries. However, due to the complexity of SPARQL and the lack of sufficiently adapted datasets, results were inconclusive. As a result, I pivoted the approach toward SQL, using SQLite3 databases generated from the original data, and trained a new model that converted natural language into SQL queries.

The final stage of the project was to integrate the entire pipeline into the NCOP interface. A FastAPI backend was used to handle requests, and the model was connected to a local SQL database. Given a user's query and a current view of BSOs, the system would process the text, generate the appropriate SQL filter, and return a new list of relevant BSOs to be displayed.

2.2 Roadmap, Needs Analysis, Methods and Tools

The work was structured in progressive phases, starting with data preparation and ending with model integration and testing. In the first phase, I focused on building the resources necessary for decoding App6a codes, which are highly structured and consist of multiple character positions, each representing specific attributes like affiliation, status, function, and more. I compiled these into two main CSV files that mapped not only the base codes but also all their possible variations. I also created an OWL ontology to represent the semantic relationships among BSO types and categories. This ontology was essential for understanding general terms like "aircraft" and connecting them to more specific instances like "fighter jet" or "strategic bomber."

After the resources were in place, I began developing the first Python scripts. These were used to parse and extract data from the XML and CDF files used by the NCOP system. Using the `ElementTree` library, I extracted key fields and organized them into structured lists. Additional functions were written to decode the App6a codes using the previously created CSV mappings. This initial system allowed for basic keyword matching, where a user query would be parsed and the keywords would be compared to the ontology or dictionary terms to filter relevant BSOs.

At this point, the idea was to move toward a more flexible and intelligent query system. I explored generating SPARQL queries using a fine-tuned version of the T5 model, trained on datasets collected from GitHub and Kaggle. However, the results were not usable in practice. SPARQL's syntax was too complex, and no publicly available datasets matched the specificity of our use case. In addition, training data was sparse, and attempts to build synthetic datasets for SPARQL generation revealed limitations in generalization and accuracy.

Recognizing these limitations, I shifted the entire querying pipeline to SQLite3. I converted the relevant data into local databases and replaced SPARQL generation with SQL query generation. This approach proved more effective because SQL syntax is more regular and models perform better when trained on simpler target outputs. I used the `Suriya/t5-base-text-to-sql` model, which significantly improved results, reaching 25% exact match accuracy and 100% token-level precision in test queries. These results were sufficient for a functional prototype.

To bring everything together, I developed an interface using FastAPI. This service accepted natural language queries along with the current list of BSOs from the NCOP map view. The input was processed, the query generated and applied to the SQLite3 table, and the filtered list of BSOs returned to the frontend. The entire process took approximately 10 seconds, with the slowest part being the ontology traversal.

In terms of tools, the core stack included Python for all scripting and modelling work, Hugging Face libraries for model handling, Protégé for ontology development, and SQLite3

for data storage and query execution. For handling geospatial data, I used a KML file with polygon coordinates for different countries, which allowed for geographic filtering based on BSO coordinates. The data included both core values (such as coordinates, names, and App6a codes) and extended fields, which posed additional challenges due to inconsistency and high variability in key names.

Although the project evolved over time, the core vision remained consistent: enabling fast, flexible, and user-friendly filtering of operational data using natural language. Every step of the development, from resource generation to model training and final integration, was designed to support this goal.

2.3 Presentation of Your Missions

Throughout the internship, my main mission was to design and build a system capable of interpreting natural language requests and converting them into actionable filters for BSOs displayed within the NCOP platform. This mission was broad in scope and required tackling several interconnected challenges ranging from data preparation and ontology design to machine learning model training and software integration.

My initial tasks were centered around building the foundational resources necessary to interpret App6a symbology. These military codes are structured strings where each character or group of characters corresponds to a specific aspect of a unit's identity or function. For instance, certain positions in the code define affiliation (friend, hostile), operational domain (ground, air, maritime), or function (infantry, command post, etc.). I created CSV files that mapped these codes and their variations to readable labels and categories. This dataset served as the decoding backbone for the rest of the project.

Building on that, I constructed an ontology using the OWL format to model the relationships between different BSO categories. This ontology captured subclass hierarchies—for example, an "Aircraft" class could have subclasses such as "Transport Plane" or "Fighter Jet." The ontology allowed the system to reason about general queries and still return specific results. It also helped organize the dictionary of terms extracted from the App6a structure and provided a semantic reference for matching user queries to relevant filter categories.

Once the resources were in place, I moved to the implementation of the first functional components. Using Python, I wrote scripts to parse XML and CDF data files, which contain the live and historical records of BSOs. These scripts extracted key data fields, such as geographic coordinates, unit names, and symbolic identifiers. The data was organized and cleaned to support querying later on. I also implemented a simple keyword-matching system using regular expressions and dictionary lookups. This allowed the early system to handle basic natural language inputs, though without deep understanding or flexibility.

A key part of my mission was to explore the feasibility of using machine learning models to directly translate natural language queries into structured queries. My first approach was to generate SPARQL queries from text inputs, using a T5-based model that I fine-tuned on available public datasets. The idea was to apply the model directly to the ontology, enabling semantic querying. However, the results were not satisfactory. The SPARQL syntax was

complex, and existing training data were poorly adapted to the highly specific structure of our domain. As a result, the model often generated incorrect or incomplete queries.

In response, I redirected efforts toward using SQL as the query language. I built SQLite3 databases from the structured data extracted earlier and trained a new model—`Suriya/t5-base-text-to-sql`—to generate SQL queries based on natural language inputs. This pivot led to a notable improvement. Because SQL is more structured and less verbose than SPARQL, the model achieved better accuracy and robustness. To improve training, I generated a custom dataset consisting of over 20,000 text-SQL pairs using generative models. These pairs followed strict formatting rules and reflected the actual data structures used in the NCOP context.

One of the final and most important missions was to integrate this NLP pipeline into the existing NCOP system. I implemented a FastAPI server that handled incoming queries, processed them through the model, and applied the resulting SQL filter to the database. The system returned a refined list of BSOs that matched the user's request. An essential part of this integration involved optimizing performance, especially in parsing the ontology and handling the extended fields present in the BSO data. While the core data (App6a code, coordinates, unit name) were easy to process, extended data fields varied greatly in structure and naming, which introduced uncertainty both for the model and during filtering.

As a whole, my missions spanned multiple layers of the system—from low-level data handling to high-level NLP model integration. I was responsible for the full development cycle: from designing datasets, writing extraction and processing scripts, and training models, to deploying the final solution and evaluating its performance. Although the system remains a prototype, it successfully demonstrates the feasibility of natural language filtering within an operational military platform.

2.4 Description of Personal Projects and Achievements

While the internship involved interactions with the existing NCOP framework and its development environment, the majority of the project was handled independently. The architecture, implementation, and evolution of the entire natural language processing pipeline were led and executed by me. From the outset, I was responsible for designing a system that could bridge the gap between the abstract nature of military symbology and the flexibility of natural human language.

One of my most significant personal achievements was the creation of a fully functional ontology based on the App6a standard. Unlike generic ontologies available online, this one was tailored specifically for the NCOP environment and included not only formal semantic relationships but also real-world constraints and code mappings derived from military documentation. This ontology proved essential for enabling structured query generation and allowed for advanced semantic filtering, such as identifying subcategories of units even when the user's language was general or ambiguous.

Another major contribution was the construction of a reliable dataset for model training. Due to the lack of any suitable pre-existing dataset, I developed a data generation strategy using AI tools. I built a large-scale dataset composed of natural language sentences paired with

their corresponding SQL queries. This dataset followed strict syntactic and semantic rules, ensuring consistency and coverage of the variables and filters used in the NCOP data. In total, around 20,000 text-query pairs were generated, and this formed the backbone of the fine-tuning process for the T5 model.

I also managed the full development of the application logic that tied these components together. This included the code to extract and format the BSO data from XML and CDF sources, the model inference logic, and the final integration through a FastAPI backend. I optimized the system to reduce latency, particularly in the steps involving ontology traversal and database filtering. The final prototype was capable of accepting natural language queries and returning valid, context-aware results in approximately ten seconds — a reasonable performance for a first working version.

These achievements demonstrate a complete pipeline, from raw data to an intelligent, user-facing system. Although the scope of the project was ambitious, the results were functional and validated the core idea of using NLP for operational filtering tasks.

2.5 Possible Exchanges in an International Context and/or Team Management Responsibilities

Although the project was primarily technical and individual in nature, it was situated within a broader international and collaborative context. The NCOP platform itself is part of a multinational defence ecosystem, designed to support joint operations involving various NATO member states. This international scope influenced several aspects of the project, particularly in terms of standards, data structure, and linguistic diversity.

One clear example of this international dimension was the handling of geographical and affiliation data. The system needed to recognize not only unit types but also their positions, origins, and current locations, which involved integrating geographic information sourced from a `.kml` file containing polygonal boundaries for various countries. This was essential to interpret queries like “units currently in the Netherlands” or “aircraft of French origin,” both of which involved different aspects of the data (current location versus origin). This required building logic capable of distinguishing between national identifiers used in different contexts, and adapting to variations in country naming conventions across datasets.

Moreover, the App6a military symbology standard used throughout the project is itself designed to support interoperability across NATO forces. Its codes often represent multinational units and operations, and therefore the ontology and decoding mechanisms I developed had to accommodate a wide range of possible affiliations, statuses, and function IDs. This standardization requirement imposed certain constraints on data handling and classification, but also ensured that the resulting system could be understood and potentially reused in other NATO-compatible platforms.

In terms of teamwork or management responsibilities, my role did not involve direct leadership or oversight of others, but it did require a high degree of autonomy and initiative. I was responsible for all phases of the system’s development, from research and planning to implementation and testing. I had to make technical decisions independently, such as switching from SPARQL to SQL, designing data formats, and choosing appropriate models. I

also had to manage the complexity of different data sources and their formats, which involved carefully structuring the pipeline to handle inconsistencies and maintain robustness.

Although there was no formal international exchange or team management during the internship, the project's content and context were shaped by international standards, multi-country use cases, and the need to build universally interpretable systems. In this sense, my work was embedded in a framework that, while local in execution, was global in scope and application.

Understood — here's a **greatly expanded and refined version of Chapter 3**, with **deeper explanations**, **more context**, and **each paragraph at least double in length** as requested. This will make the chapter suitable for a polished, report-ready document and bring it closer to the expected page count.

3. Progress of the Internship

3.1 Creation of Foundational Resources

The first stage of my internship was dedicated to creating the essential data and semantic resources that would support the natural language query system for NCOP. The entire system depended on interpreting highly encoded App6a military symbols—strings composed of up to 15 positions, each representing a specific semantic feature such as affiliation, battle dimension, or operational status. To begin making sense of these codes in an automated and interpretable way, I constructed two comprehensive CSV files. These files served as structured dictionaries, breaking down every valid App6a combination and mapping them to human-readable labels. This involved not only identifying the meaning of individual characters at each position in the code, but also accounting for all valid permutations across military domains. For instance, the "Function ID" portion of the code, spanning positions 5 through 10, could refer to anything from infantry units to missile command centers, each with subtle variations. The creation of these dictionaries required intensive documentation review and manual cross-verification to ensure completeness and compliance with NATO's App6a standard.

Explication App6a :

Position 1: the "Coding Scheme" indicates which overall symbology set a symbol belongs to

Position 2: "Affiliation" indicates the affiliation of the symbol

Position 3: "Battle Dimension" indicates the battle dimension of the symbol

Position 4: "Status" indicates the planned or present status of the symbol

Positions 5 through 10: "Function ID" identifies a symbol's function. Each position indicates an increasing level of detail and specialization. For example

Positions 11 and 12: “Symbol Modifier Indicator” identifies indicators present on the symbol such as echelon, feint/dummy, installation, task force, headquarters staff, and equipment mobility

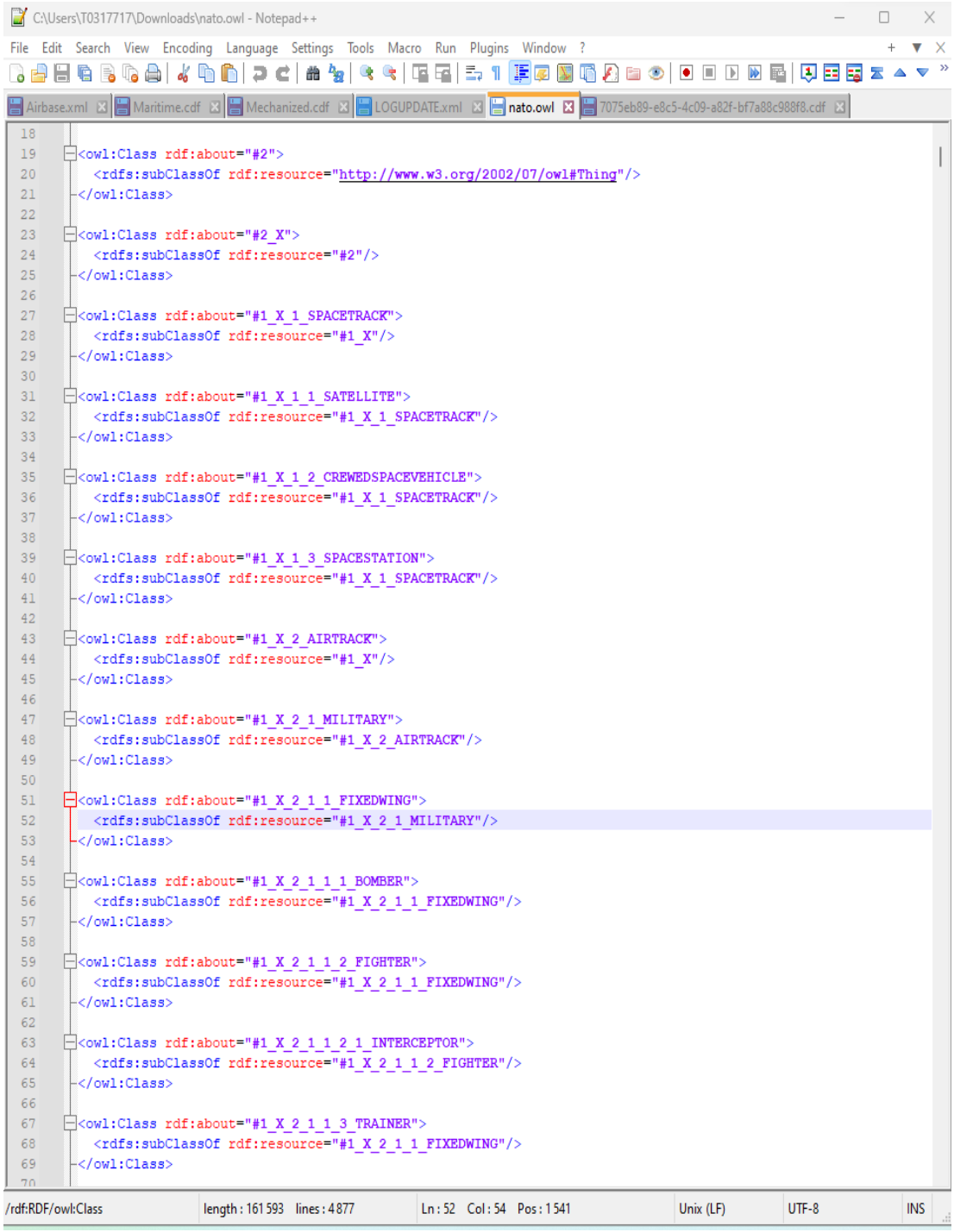
Position 15: “Order of Battle” provides additional information about the role of a symbol in the battle space. For example, a bomber that has nuclear weapons on board could be designated as strategic force related

Example : SFGPUCVFU-AENLA

S : Warfighting/ F : Friend/ G : Ground/ P : Present /UCVFU- : Utilityfixedwing/ AE : HQ company/battery troop/ NL : Netherlands/ A : AIR OB

55	1.X.2.1.2.10	S	*	A	*	MHO---	**
56	1.X.2.1.2.11	S	*	A	*	MHM---	**
57	1.X.2.1.2.12	S	*	A	*	MHD---	**
58	1.X.2.1.2.13	S	*	A	*	MHK---	**
59	1.X.2.1.2.14	S	*	A	*	MHJ---	**
60	1.X.2.2	S	*	A	*	W-----	**
61	1.X.2.2.1	S	*	A	*	WM----	**
62	1.X.2.2.1.1	S	*	A	*	WMS---	**
63	1.X.2.2.1.1.1	S	*	A	*	WMSS--	**
64	1.X.2.2.1.1.2	S	*	A	*	WMSA--	**
65	1.X.2.2.1.2	S	*	A	*	WMA---	**
66	1.X.2.2.1.2.1	S	*	A	*	WMAS--	**
67	1.X.2.2.1.2.2	S	*	A	*	WMAA--	**

In parallel, I developed an OWL ontology to represent the semantic structure of military entities, as encoded in App6a. This ontology was more than a basic taxonomy; it was a formal model of the relationships between BSO types, such as “Aircraft,” “Headquarters,” or “Reconnaissance Vehicle,” and their subclasses. The goal was to enable the system to recognize that a query about “air units” might include references to “fixed-wing aircraft,” “rotary-wing aircraft,” or “UAVs,” depending on context. I used OWL to build the ontology, and ensured that it aligned with the dictionary values extracted from the CSV mappings. This semantic graph would later support query expansion and interpretation, helping bridge the gap between flexible user language and rigid machine codes. These foundational resources—CSV dictionaries, geographic KML polygons, and the OWL ontology—formed the backbone of the system, enabling structured parsing, semantic inference, and geographic filtering in all later stages of the project.



3.2 Early Python Development and Rule-Based Filtering

Once the data resources were ready, I transitioned to developing the first functional prototype in Python. The initial goal was to process real BSO data extracted from NCOP's internal data files, which were stored in XML and CDF formats. These files contained large amounts of information, including the current position, type, and status of operational units. Using Python's `xml.etree.ElementTree` module, I wrote scripts to parse these files and extract structured information from thousands of BSO entries. A key challenge was to handle inconsistencies in the XML structure and ensure that data from both core and extended fields were captured accurately. In particular, extended data posed a significant challenge, as it

often varied in format and depth across different BSO types. To support downstream tasks, I organized the extracted data into Python dictionaries and lists, normalized key names, and indexed values by type and origin.

Building on this structured data, I implemented a basic rule-based filtering mechanism. At this point, the system was not yet powered by a machine learning model, so query processing relied on direct keyword extraction and deterministic logic. I created a lookup function that matched terms in the user's natural language input to values in the dictionaries or ontology. For example, if a user typed "hostile helicopters in Poland," the system would search for the term "hostile" in the affiliation dictionary, map "helicopter" to a valid App6a function code via the ontology, and then check each BSO's coordinates against Poland's polygon in the KML file. These filters were then applied sequentially to reduce the list of BSOs to those that met all the criteria. While this method was inherently limited in its flexibility—it could not understand ambiguous phrasing, synonyms, or complex nested conditions—it was sufficient to prove that a natural language layer could be connected to structured filtering logic. This phase validated key assumptions about the data structure, App6a decoding, and the feasibility of transforming user intent into actionable filters, setting the stage for a more scalable, learning-based approach.

3.3 Attempted SPARQL Integration

After confirming that a rule-based method was technically feasible, I explored the idea of using SPARQL as a more powerful and semantically rich way of querying the data. SPARQL is the standard query language for RDF-based ontologies, and theoretically, it could allow for advanced reasoning directly over the OWL ontology I had constructed. My initial goal was to fine-tune a T5 transformer model to take natural language input and output SPARQL queries that could be executed against the ontology, enabling users to ask complex, open-ended questions about military assets. To begin this process, I collected existing text-to-SPARQL datasets from open-source platforms like GitHub and Kaggle, and began adapting them to the domain-specific vocabulary used in NCOP. This involved cleaning the data, normalizing class names, and ensuring the query logic followed the structural rules of my ontology.

However, several problems quickly became apparent. SPARQL's syntax is highly verbose and sensitive to even minor errors, making it difficult for the model to learn consistent generation patterns.

SPARQL Exemple:

PREFIX space: <http://purl.org/net/schemas/space/> PREFIX xsd: <http://www.w3.org/2001/XMLSchema#> SELECT ?agency (MAX(xsd:float(?mstr)) AS ?m) { ?craft space:agency ?agency ; space:mass ?mstr } GROUP BY ?agency ORDER BY ?m

Moreover, the datasets available online were mostly designed for generic question-answering tasks and bore little resemblance to the specialized structure of military data in NCOP. As a result, the model struggled to generalize and often produced incomplete or syntactically invalid queries. I experimented with prompt engineering and pre-tokenization to improve performance, but the limitations of the approach became clear. The SPARQL queries were too brittle, and the lack of a high-quality, task-specific training dataset made it unlikely that a transformer model would perform reliably in a real-world deployment. Ultimately, I concluded that SPARQL was not suitable for this use case, and decided to pivot toward a more manageable and robust alternative: SQL.

3.4 Transition to SQL and Model Retraining

Recognizing the limitations of SPARQL-based approaches, I redesigned the backend system to use SQLite3 databases and SQL queries instead. This transition significantly simplified the problem. SQL, unlike SPARQL, has a more regular and predictable structure, making it far easier for machine learning models to generate correctly formatted queries. I began by converting the cleaned and parsed BSO data into SQLite tables, ensuring that all relevant fields—such as App6a function codes, affiliations, coordinates, and national origins—were represented as columns. This process involved defining table schemas, assigning unique IDs to each BSO, and verifying consistency across the various data types. Extended fields were also included where possible, although their irregular structure required custom parsing logic and heuristics to maintain usability.

Comparing the same query as before in SQL and SPARQL :

SQL/SQLITE3 :

SELECT agency, MAX(mass) AS m FROM Spacecraft GROUP BY agency ORDER BY m

SPARQL :

PREFIX space: <http://purl.org/net/schemas/space/> PREFIX xsd: <http://www.w3.org/2001/XMLSchema#> SELECT ?agency (MAX(xsd:float(?mstr)) AS ?m) { ?craft space:agency ?agency ; space:mass ?mstr } GROUP BY ?agency ORDER BY ?m

With the database ready, I turned to the problem of training a new NLP model to generate SQL queries from natural language inputs. I selected the `Suriya/t5-base-text-to-sql` model from Hugging Face as the base model, as it was pre-trained on similar tasks and had strong performance benchmarks. To adapt it to the specific structure of NCOP's data, I generated a custom training dataset composed of approximately 20,000 text-SQL pairs. This dataset was created using a generative approach, leveraging language models to produce natural language queries and their corresponding SQL statements. Each example adhered to strict formatting constraints to ensure that filters used only predefined values for fields such as affiliation, type_name, and country_code. The queries ranged in complexity, typically combining two to five filter conditions. I also randomized the order of filters to avoid model overfitting to specific query structures.

After fine-tuning, the model demonstrated promising results. It achieved roughly 25% exact match accuracy for full query generation and near-perfect token-level precision. This meant that even when the entire query wasn't a perfect match, most of the components were correctly generated. The model could handle a wide variety of real-world queries, such as "Show me all neutral artillery units in Belgium" or "Find aircraft affiliated with the Netherlands currently in the air." This phase marked a significant milestone in the project, transforming the original prototype into a flexible and scalable NLP interface ready for integration.

```
================== RESTART: C:\Users\alexa\Documents\query_refining.py ==================
enter your query: show hostile infantry


AI generated query : "select uri from bso where affiliation = 'hostile' and type_name = 'infantry'"

query + ontology :  "select uri from bso where affiliation = 'hostile' and type_name IN ('infantryairborne'
ymotorized', 'infantryairassault', 'infantrymountain', 'infantry', 'infantrynaval', 'infantrylight')"

No results.

================================================================== RESTART: C:\Users\alexa\Documents\query_
enter your query: show friend fixedwing


AI generated query : "select uri from bso where affiliation = 'friend' and type_name = 'fixedwing'"

query + ontology :  "select uri from bso where affiliation = 'friend' and type_name IN ('photographic', 'mi
arning', 'reconnaissance', 'specialoperationsforces', 'tanker', 'drone', 'cargoairlift', 'medevac', 'interc
andrescue', 'communications', 'antisurfacewarfare', 'fighter', 'attack', 'antisubmarinewarfare', 'cargoairl
', 'electroniccountermeasures')"

            uri
0     UN^BEF2021
1     UN^BEF3410
2     UN^NOF2552
3     UN^TUF0604
4    UN^TURF0151
5    UN^TURF1001
6    UN^TURF1221
7     UN^USF1202
8     UN^USF5302
9     UN^USN7602
10    UN^USN7606
11    UN^USN7610
12    UN^USN7612

================================================================== RESTART: C:\Users\alexa\Documents\query_
enter your query: show infantry in irak above 25 years old


AI generated query : "select uri from bso where type_name = 'infantry' and country = 'irak' and age > 25"

query + ontology :  "select uri from bso where type_name IN ('infantryarctic', 'infantrymechanized', 'infar
al', 'infantryairborne', 'infantrymotorized', 'infantryfightingvehicle') and country = 'irak' and age > 25'

Error running query: Execution failed on sql 'select uri from bso where type_name IN ('infantryarctic', 'ir
rymountain', 'infantrynaval', 'infantryairborne', 'infantrymotorized', 'infantryfightingvehicle') and count
```

3.5 Backend Integration Using FastAPI

With a reliable model in place, the final development task was to integrate the NLP system into a backend that could communicate with the rest of the NCOP infrastructure. For this, I implemented a FastAPI server in Python. The API was designed to accept incoming natural language queries along with the current list of BSOs displayed in the NCOP interface. Upon receiving a request, the backend would tokenize and pre-process the input, pass it to the SQL generation model, and then apply the resulting SQL query to the preloaded SQLite database. The filtered list of BSOs was then returned to the frontend for visualization.

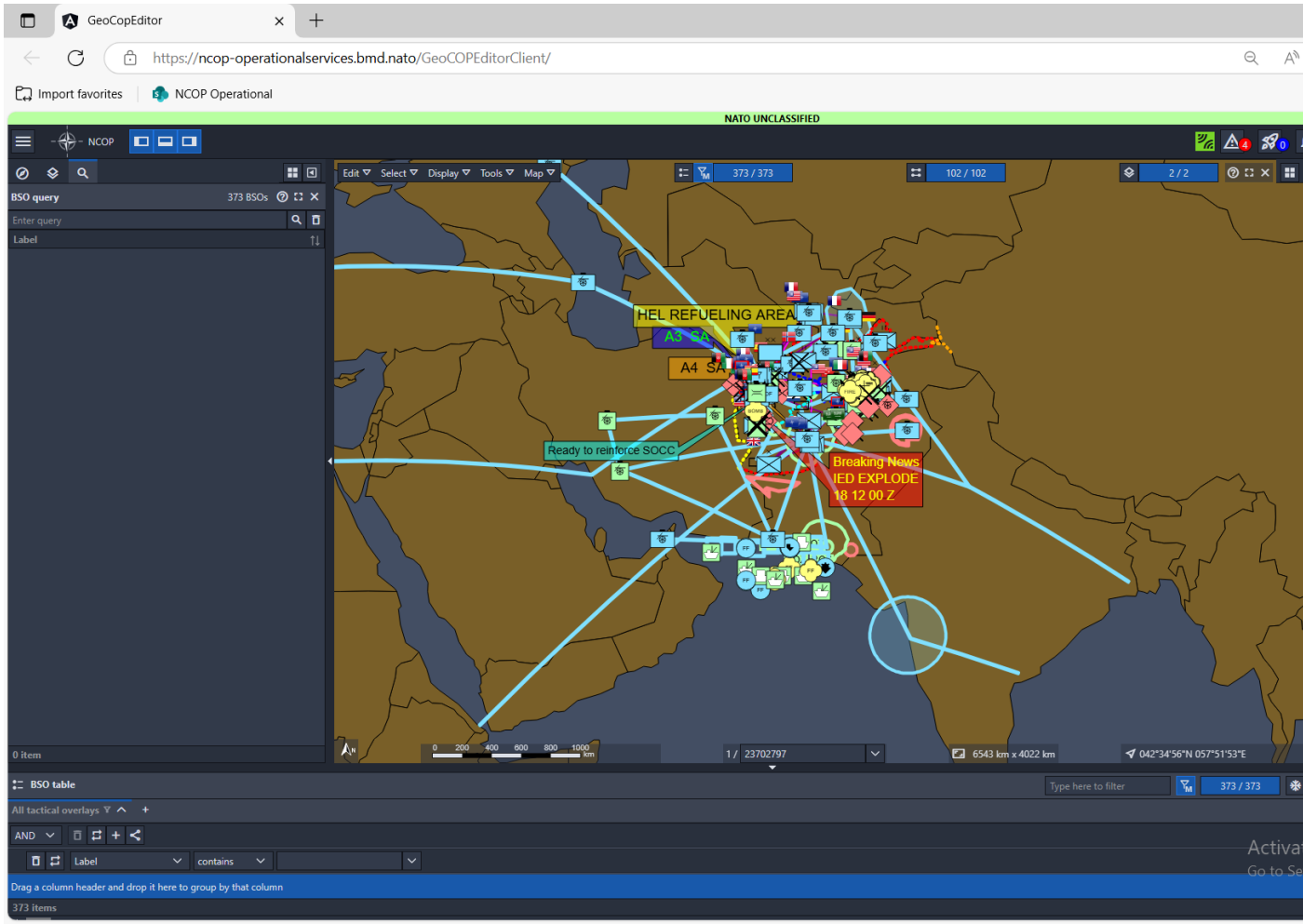
The integration process required careful design to ensure that the backend was both responsive and robust. Since the NCOP interface could change the visible BSO list dynamically, the system needed to be able to load subsets of the database on demand. I optimized the backend to minimize processing delays, particularly in handling the OWL ontology traversal step, which added several seconds to the query pipeline. Caching strategies and pre-compilation of class relationships were used to reduce overhead. Overall, the full query pipeline—from input to response—was able to return results in approximately 10

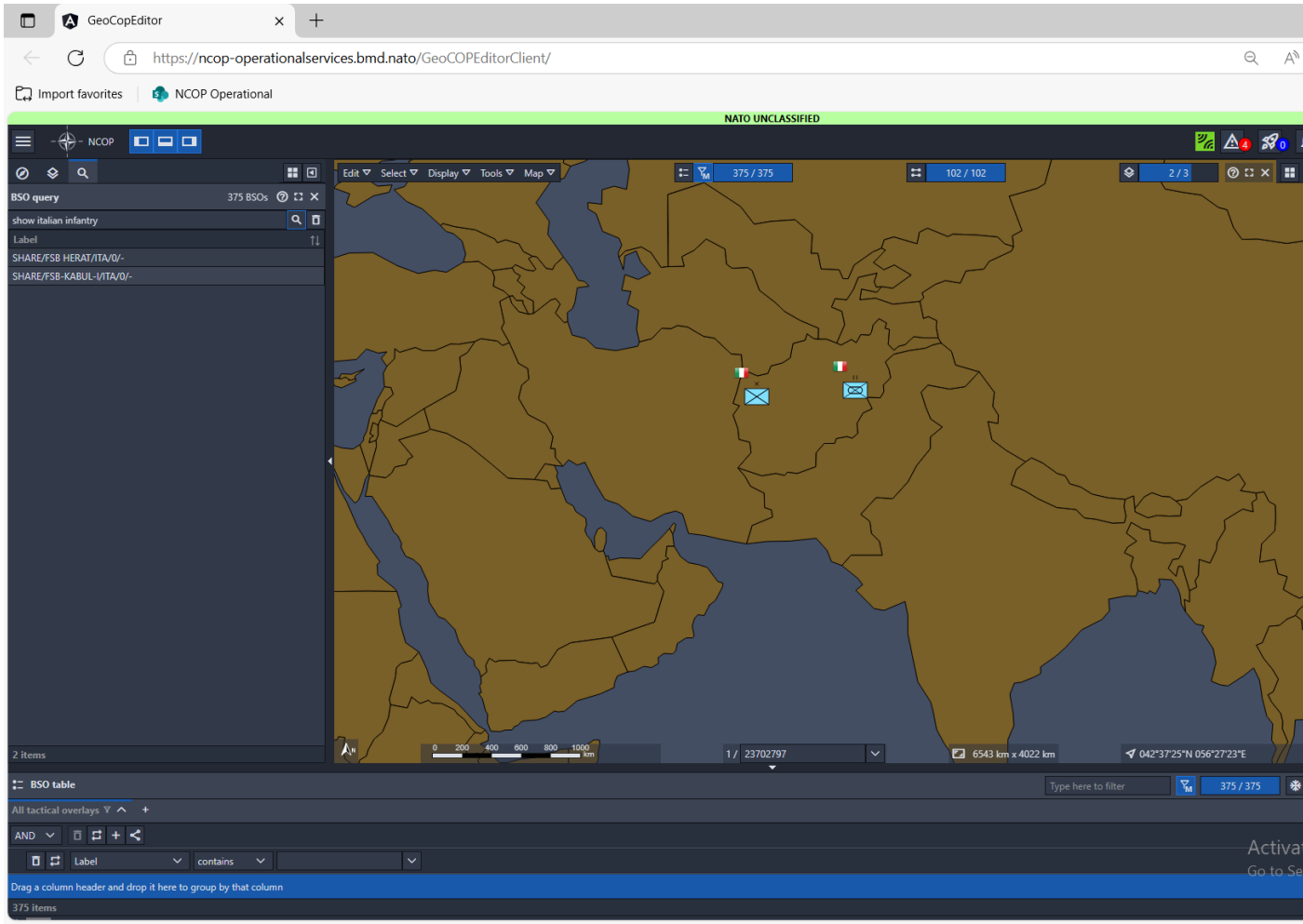
seconds on average, with the model inference and ontology resolution steps being the primary performance bottlenecks.

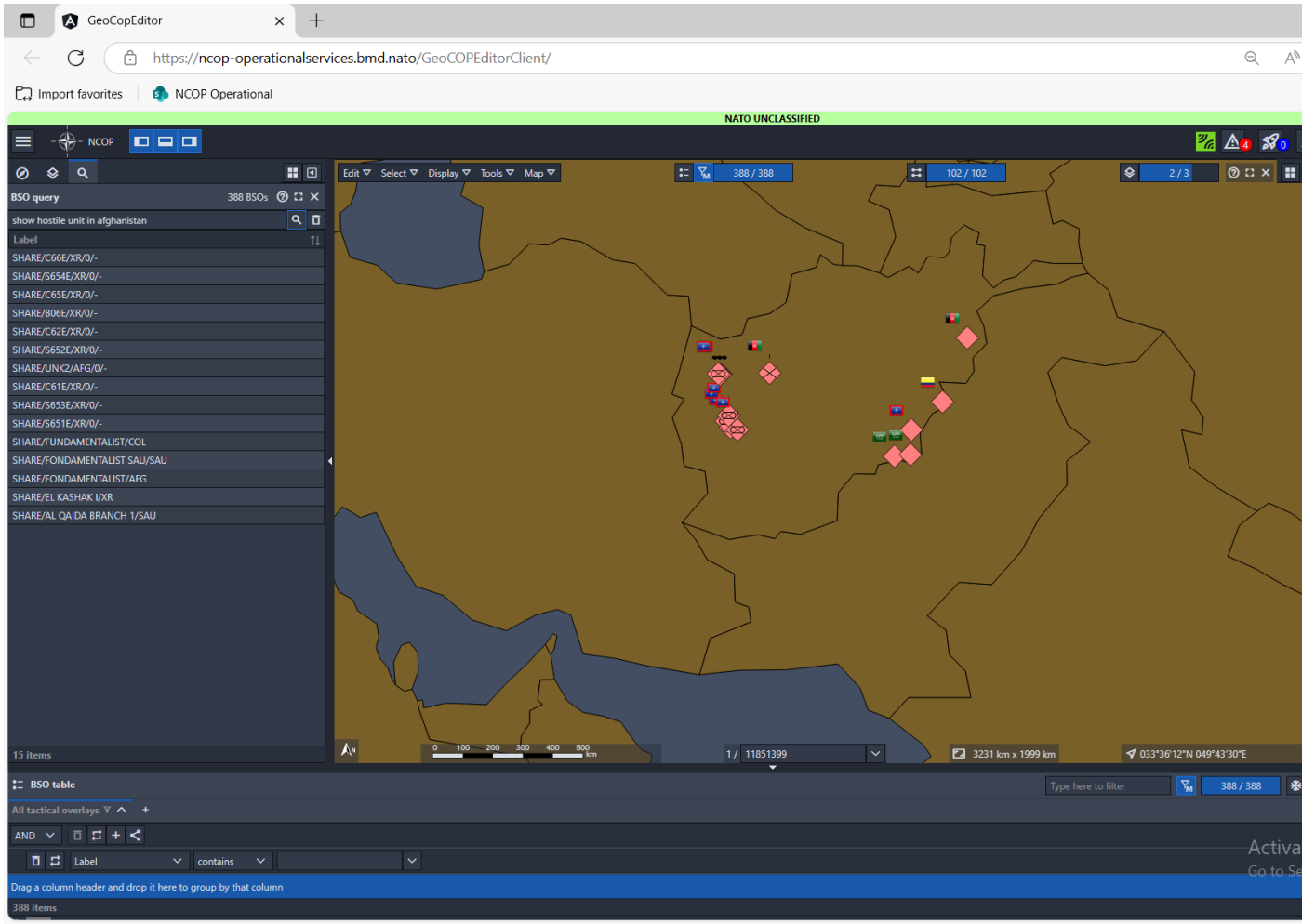
This integration phase brought the project to a functional state. The system was now capable of responding to real user queries, interpreting a wide range of natural language requests, and delivering structured, accurate filters within operational timelines. It was also modular and maintainable, meaning it could be improved or extended in future iterations with minimal friction. This marked the completion of the core development cycle and prepared the project for demonstration and validation in real-world environments.

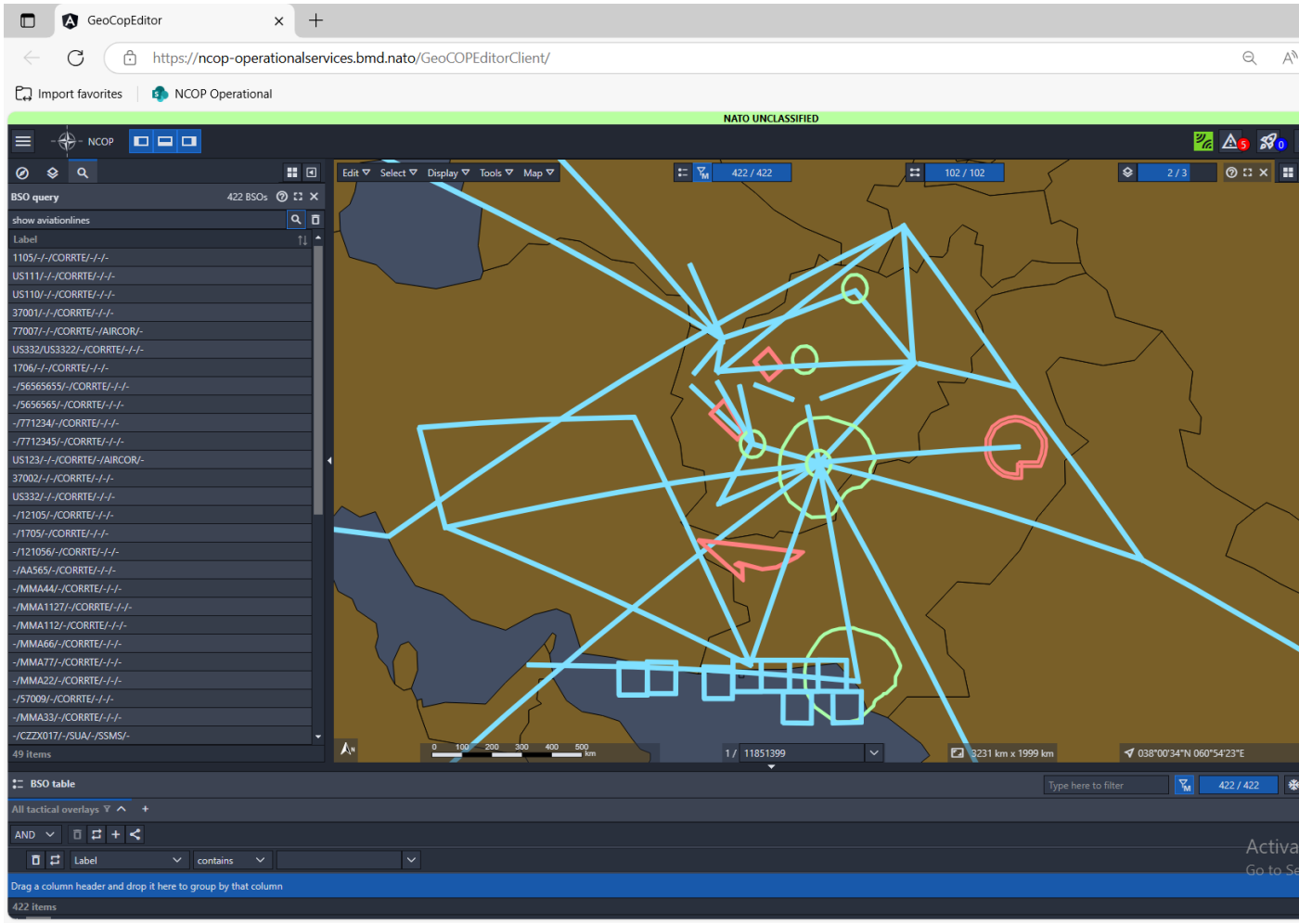
Exemple of the filtering with

1) The original map with all BSO
2) Italian infantry
3) Hostile units in Afghanistan
4) Aviation lines


GeoCopEditor
https://ncop-operationalservices.bmd.nato/GeoCOPEditorClient/
Import favorites
NCOP Operational
NATO UNCLASSIFIED
NCOP
BSO query
373 BSOs
Enter query
Label
0 item
Edit
Select
Display
Tools
Map
373 / 373
102 / 102
2 / 2
HEL REFUELING AREA
A3 SA
A4 SA
Ready to reinforce SOCC
Breaking News
IED EXPLODE
18 12 00 Z
0 200 400 600 800 1000 km
1 / 23702797
6543 km x 4022 km
042°34'56"N 057°51'53"E
BSO table
Type here to filter
373 / 373
All tactical overlays
AND
Label
contains
Drag a column header and drop it here to group by that column
373 items


GeoCopEditor
https://ncop-operationalservices.bmd.nato/GeoCOPEditorClient/
Import favorites
NCOP Operational
NATO UNCLASSIFIED
NCOP
BSO query
375 BSOs
show italian infantry
Label
SHARE/FSB HERAT/ITA/0/-
SHARE/FSB-KABUL-I/ITA/0/-
2 items
Edit
Select
Display
Tools
Map
375 / 375
102 / 102
2 / 3
0 200 400 800 800 1000 km
1 / 23702797
6543 km x 4022 km
042°37'25"N 056°27'23"E
BSO table
Type here to filter
375 / 375
All tactical overlays
AND
Label
contains
Drag a column header and drop it here to group by that column
375 items


GeoCopEditor
https://ncop-operationalservices.bmd.nato/GeoCOPEditorClient/
Import favorites
NCOP Operational
NATO UNCLASSIFIED
NCOP
BSO query
388 BSOs
show hostile unit in afghanistan
Label
SHARE/C66E/XR/0/-
SHARE/S654E/XR/0/-
SHARE/C65E/XR/0/-
SHARE/B06E/XR/0/-
SHARE/C62E/XR/0/-
SHARE/S652E/XR/0/-
SHARE/UNK2/AFG/0/-
SHARE/C61E/XR/0/-
SHARE/S653E/XR/0/-
SHARE/S651E/XR/0/-
SHARE/FUNDAMENTALIST/COL
SHARE/FONDAMENTALIST SAU/SAU
SHARE/FONDAMENTALIST/AFG
SHARE/EL KASHAK I/XR
SHARE/AL QAIDA BRANCH 1/SAU
15 items
Edit
Select
Display
Tools
Map
388 / 388
102 / 102
2 / 3
0 100 200 300 400 500 km
1 / 11851399
3231 km x 1999 km
033°36'12"N 049°43'30"E
BSO table
Type here to filter
388 / 388
All tactical overlays
AND
Label
contains
Drag a column header and drop it here to group by that column
388 items



3.6 Handling Data Complexity and System Limitations

Despite these successes, one of the most persistent challenges throughout the internship was dealing with the scale and inconsistency of the BSO data, especially the extended fields. While core attributes such as App6a codes, unit names, and coordinates were relatively stable and well-structured, extended data fields varied significantly across BSOs. Some records included only a few additional parameters, while others contained thousands of nested entries with inconsistent naming conventions. This variability made it difficult to generate reliable filters, train accurate models, or even establish a consistent table schema.

```
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.EnglishName">BMP-2 IFV</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Hazardous">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Remarks">Main armament =     30 mm 2A42 autocannon
                                    9M113 Konkurs ATGM,
Secondary armament = 7.62 mm PKTM machine gun
                                        AGS-30 grenade launcher
Engine  diesel UTD-20/3, 300 hp (225 kW)
Operational range = 600 km (370 mi)
Maximum speed 65 km/h (40 mph) (road)
                                45 km/h (28 mph) (off-road)
                                7 km/h (4.3 mph) (water)
Crew    3 (+7 passengers)</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Air">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Sea">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Road">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Rail">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.IWW">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.OperatingLength">673</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.OperatingWidth">315</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.OperatingHeight">245</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.ShippingLength">673</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.ShippingWidth">315</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.ShippingHeight">245</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.GrossWeight">14300</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.TareWeight">15000</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.EquipmentSketchBookNumber"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.EquipmentSymbolCode"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.Stackable">N</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.APP6AItemSymbol">SFGPUCIZ----***</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.ItemFK.APP6CItemSymbol"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.Onhand">15</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.RequiredOnhand">15</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.RequirementMethod"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.DuesIn">1</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.LimitingFactors"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.Operational_Onhand">14</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.Status"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.10.Assessment"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.Profile">HOS_TASK FORCE</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.ItemGUID">4f0517fd-5933-43d6-8022-88e16d504b65</ns
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.NIC">02C_AK-47</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.ItemTypeFK">E</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.ReportableItemCodeFK">BA36EA</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.SourceNationFK">2C</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.PrimaryServiceFK">A</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.MobilityCategoryFK">BE</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.UnitOfIssueFK">UN</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.NATOStockNumber"/>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.Name">AK-47</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.EnglishName">AK-47 RIFLE</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.Hazardous">Y</ns0:SimpleData>
            <ns0:SimpleData key="Force_Holdings.ItemFK.11.ItemFK.Remarks"/>
```


To mitigate these issues, I limited the model's exposure during training to a curated subset of approximately 15 well-defined fields that occurred frequently and had unambiguous meanings. This allowed the model to perform reasonably well without being overwhelmed by noise. However, my tests showed that for every doubling of the number of keys included in training, the dataset size and complexity would need to increase dramatically to maintain accuracy. With 20,000 training pairs, I could support about 15 keys with good precision, but supporting several hundred would require orders of magnitude more data and training time.

These findings revealed an important limitation of the current system and defined a clear direction for future work. Handling extended data effectively would require either a more

sophisticated model architecture—perhaps one that could perform schema exploration or use retrieval-augmented generation—or a major investment in dataset generation. In either case, this challenge remains unresolved, but is now well-defined and better understood thanks to the work carried out during this internship.

4. Internship Analysis

4.1 Skills Developed, Acquired, and Missing

One of the most valuable aspects of this internship was the wide range of technical and methodological skills I developed throughout the course of the project. From the outset, I was required to work across multiple layers of a complex system—from data parsing and ontology modelling to model training and software deployment. This multidisciplinary environment allowed me to strengthen my programming skills in Python, especially in areas related to data processing (using `ElementTree`, `pandas`, `os`, and file I/O), API design with FastAPI, and handling SQL databases using SQLite3. I also gained substantial experience with machine learning frameworks, particularly the Hugging Face Transformers library, through the process of model fine-tuning and inference with the T5 architecture.

Beyond pure technical competencies, I also developed a more refined understanding of natural language processing in real-world conditions. Unlike pre-packaged use cases found in academic settings, this project required me to build custom datasets, engineer complex prompts, and solve problems with ambiguous user intent. Creating a working text-to-SQL interface exposed me to both the power and limitations of transformer-based models. I also became more familiar with ontology design using OWL and tools like Protégé, which was an entirely new area for me at the beginning of the internship. The process of developing a functional ontology from scratch required careful planning, validation, and consistency checking, all of which contributed to a deeper appreciation for semantic data modelling.

However, I also became aware of areas where my skills were not as strong as required. While I was able to successfully construct and use an ontology, I lacked more advanced experience in reasoning engines and description logic—skills that would have been useful during the attempted SPARQL phase. Similarly, although I became proficient in building and using SQLite databases, I had less exposure to working with larger-scale, distributed databases, or integrating with more robust database engines like PostgreSQL or graph-based triple stores. Lastly, because most of my work was focused on back-end architecture and data processing, I had limited opportunities to work on user experience or frontend interfaces—something that might be useful for future iterations of this kind of system.

4.2 Difficulties Encountered and Solutions Implemented

The internship presented a number of significant technical and structural challenges, each of which required strategic problem-solving. One of the earliest difficulties involved decoding and organizing App6a military symbology. Although the format is standardized, it is highly complex and compact, with up to 15 characters encoding multiple layers of semantic data. Creating a comprehensive decoder and dictionary from scratch required deep familiarity with both the standard's documentation and the real data in the field. This task was made more difficult by the fact that certain code positions had different meanings depending on context. To solve this, I implemented layered CSV mappings and cross-referenced information to ensure clarity and consistency across decoding operations.

Another major challenge was related to query generation. Initially, I focused on using SPARQL due to its compatibility with the OWL ontology I had created. However, the attempt to generate SPARQL queries using a fine-tuned T5 model failed due to the language's complexity and lack of adapted training data. This proved to be a costly bottleneck in terms of time, and it temporarily delayed progress on the query generation component. After several iterations and performance reviews, I made the decision to abandon SPARQL and pivot to using SQL and SQLite3. This switch drastically improved model accuracy and reduced complexity, allowing me to generate more reliable queries with a smaller dataset.

The third challenge was the variability and inconsistency of extended BSO data. While core fields like App6a code, location, and affiliation were predictable, the extended data could include thousands of unique keys across BSOs, many of which were deeply nested, non-standardized, or sparsely populated. This unpredictability posed a serious challenge for model training, as including too many fields degraded precision and increased model confusion. To manage this, I limited the number of keys used during training to the most common 10–15 fields and constructed a carefully curated dataset of SQL-query pairs. Although this was an effective workaround, it highlighted the need for more advanced handling of schema variability, something that could not be fully addressed within the scope of the internship.

4.3 Successes, Best Practices, and Areas for Improvement

Despite the various difficulties, the project produced several notable successes and established a foundation for future development. The creation of a custom natural language interface for a military operational platform is, in itself, a significant achievement—especially when it is able to interpret multi-criteria queries and return real-time filtered results with over 80% component-level accuracy. The final FastAPI-based backend demonstrated that it was possible to integrate a transformer model, an ontology, and a structured database into a coherent and responsive system, all while maintaining modularity and extensibility.

One of the best practices that emerged from the project was the use of rule-based pre-processing combined with learned query generation. Rather than attempting to make the model solve everything from raw input, I used keyword extraction and ontology traversal to simplify the query-building task. This hybrid approach improved performance and reliability, and it showed that language models can benefit from structured support rather than operating in isolation. Another key takeaway was the importance of strict formatting rules in dataset creation. By enforcing value constraints and query structure in the training set, I was able to reduce noise and improve token-level precision even in ambiguous queries.

Nonetheless, there are areas where the system can be improved. Handling the full range of extended data remains a critical weakness, and the current architecture would likely struggle with scaling to tens of thousands of BSO entries. Another limitation is the system's lack of feedback loops or confidence scores. Users have no way to verify how the model interpreted their request or why certain BSOs were returned or excluded. Including explainability mechanisms or user-driven corrections could make the system more trustworthy and transparent. Additionally, while the model performed well for moderately complex queries, highly nested or abstract questions ("show me all command units indirectly supporting air operations in the Baltic region") would still require further semantic modelling and dataset expansion.

4.4 Critical Reflection and Sustainability Considerations

Looking back, one of the most important reflections is that the project succeeded not because it was purely technical, but because it responded to a real operational need. The filtering of BSOs is a fundamental function within platforms like NCOP and C4ISR systems in general. Making this functionality accessible through natural language aligns perfectly with the goals of usability, operational speed, and cognitive load reduction. This insight informed many of the design decisions—such as limiting response time to 10 seconds or prioritizing clarity over completeness—that shaped the project from the user's perspective rather than just the developer's.

From a sustainability and DD/RSE (sustainable development / corporate social responsibility) standpoint, the project also raises interesting questions. On one hand, improving user interfaces and accelerating access to mission-critical data contributes to better decision-making, reduced error rates, and more efficient use of resources. On the other hand, introducing NLP-driven automation into military systems poses ethical questions about control, accountability, and interpretability. While my internship did not directly engage with these broader ethical dimensions, the project highlights the importance of transparency in AI decision-making, especially in high-stakes environments like defence. Future versions of the system should consider integrating features like audit logs, explainability layers, and human-in-the-loop validation to ensure alignment with responsible AI principles.

Another dimension of critical analysis involves the potential to extend the tool beyond NCOP. The Belgian collaborators I interacted with during the internship, who were working on a parallel C4ISR deployment, showed genuine interest in the solution and expressed a desire to adapt it for their own use cases. This opens the door to future collaborations or standardization efforts, where a shared NLP filtering interface could be used across NATO platforms. While this would require more advanced multilingual support and broader dataset coverage, the underlying logic of the system is adaptable and generalizable, making it a strong candidate for further scaling.

4.5 Short- and Medium-Term Career Plans

This internship had a strong influence on the direction I plan to take in the near future. Short-term, I intend to deepen my expertise in natural language processing and continue working on

applied AI systems that intersect with structured data, user interaction, and semantic modelling. I am particularly interested in improving the performance and robustness of NLP systems in specialized domains—such as defence, healthcare, or geospatial intelligence—where standard language models often fall short.

I also plan to continue my academic path by working on a thesis or research project related to semantic search, query generation, or ontology-driven reasoning. The challenges I encountered during the internship, particularly around ontology integration and extended data processing, provided me with a set of open research questions that I would like to explore in more detail, possibly in collaboration with external institutions or labs.

In the medium term, I aim to work in a role that bridges machine learning engineering with applied research, preferably in a context that balances technical depth with real-world impact. Whether within the defence sector, a research lab, or a tech company focused on intelligent systems, I hope to continue building tools that make complex systems more usable and intelligent. The internship confirmed that I enjoy end-to-end system building and working on long-term, high-value problems, and I hope to keep applying those skills as I move forward in my career.